

## List of Abbreviations

| Abbreviation | Full Name                                   |
|--------------|---------------------------------------------|
| <b>HW</b>    | Hardware                                    |
| <b>SW</b>    | Software                                    |
| <b>PULP</b>  | Parallel Ultra-Low-Power                    |
| <b>FFT</b>   | Fast Fourier Transform                      |
| <b>DFT</b>   | Discrete Fourier Transform                  |
| <b>OFDM</b>  | Orthogonal Frequency Division Multiplexing  |
| <b>MIMO</b>  | Multiple-Input Multiple-Output              |
| <b>AXI</b>   | Advanced extensible Interface               |
| <b>TCDM</b>  | Tightly Coupled Data Memory                 |
| <b>SoC</b>   | System-on-Chip                              |
| <b>FC</b>    | Fabric Controller                           |
| <b>CPU</b>   | Central Processing Unit                     |
| <b>GPIO</b>  | General Purpose Input/Output                |
| <b>UART</b>  | Universal Asynchronous Receiver-Transmitter |
| <b>I2C</b>   | Inter-Integrated Circuit                    |
| <b>SPI</b>   | Serial Peripheral Interface                 |
| <b>APB</b>   | Advanced Peripheral Bus                     |
| <b>DMA</b>   | Direct Memory Access                        |
| <b>ROM</b>   | Read-Only Memory                            |
| <b>RTL</b>   | Register Transfer Level                     |
|              |                                             |

## List of Symbols

| Symbol     | Full Name                         |
|------------|-----------------------------------|
| $N$        | FFT size / Number of points       |
| $n$        | Time-domain sample index          |
| $k$        | Frequency-domain subcarrier index |
| $x[n]$     | Time-domain input sample          |
| $X[k]$     | Frequency-domain output sample    |
| $A$        | First butterfly input             |
| $B$        | Second butterfly input            |
| $W$        | FFT twiddle factor                |
| $Y_0$      | First butterfly output            |
| $Y_1$      | Second butterfly output           |
| $r$        | FFT radix                         |
| $S$        | Number of FFT stages              |
| $j$        | Imaginary unit                    |
| $O(\cdot)$ | Algorithmic complexity notation   |
|            |                                   |
|            |                                   |
|            |                                   |
|            |                                   |
|            |                                   |
|            |                                   |
|            |                                   |
|            |                                   |
|            |                                   |
|            |                                   |

# Chapter 9

## 1 Hardware/ Software Co-Design

### 1.1 Introduction

In many signal-processing applications, a small subset of operations often dominates the overall execution time. Identifying these computational bottlenecks is a fundamental step in the hardware–software co-design process. Once identified, these critical operations can be redesigned as dedicated hardware accelerators and integrated into the system architecture to improve overall performance.

However, the effectiveness of such an approach depends not only on the computational efficiency of the accelerator itself, but also on the communication overhead introduced by the target platform and memory subsystem. As a result, accelerator design and system integration must be considered jointly to achieve meaningful performance gains. The methodology adopted for this hardware–software partitioning is presented in the chapter “**HW/SW Codesign**”, while the architectural features of the target platform are introduced in the chapters “**PULP Platform**” and “**Multi Cluster PULP**.”

The work presented in this thesis is motivated by the requirements of a MIMO-OFDM receiver, which serves as the target system investigated throughout this project. As discussed in the chapter “**MIMO in Mobile Communication Systems**,” modern wireless communication systems employ OFDM modulation to efficiently utilize the available spectrum and mitigate the effects of multipath propagation.

Within the receiver processing chain, the FFT is responsible for converting the received signals from the time domain to the frequency domain prior to channel equalization and symbol detection. In the considered system, this operation is performed repeatedly on large OFDM symbols and across multiple receive antennas, resulting in a substantial computational workload. Consequently, FFT processing becomes an attractive candidate for hardware acceleration.

To investigate the suitability of hardware acceleration for this workload, this thesis explores the design and integration of dedicated hardware accelerators for FFT butterfly computations on the PULP platform. Multiple butterfly architectures and radix implementations are developed and evaluated.

The integration of these accelerators into the PULP environment relies on the software infrastructure presented in “**Firmware**” and the architectural concepts discussed in “**PULP Platform**” and “**Multi Cluster PULP**.” Particular attention is given to the interaction between the processor, memory hierarchy, and accelerator interfaces, as these factors can significantly influence the overall system performance.

The resulting implementations are evaluated in terms of performance, communication overhead, and integration complexity. Rather than focusing solely on computational speedup, the analysis aims to assess the practical trade-offs associated with accelerator-based FFT execution within a programmable many-core platform.

The insights obtained from this study provide a foundation for the hardware architectures presented later in the thesis, particularly in “**Hardware II: HWPE**,” “**Hardware III: System Pipeline**,” and “**System Performance**”.

**Analysis.”** These chapters build upon the lessons learned from the FFT acceleration exploration to develop a more efficient system-level architecture for the target MIMO receiver.

### 1.1.1 Chapter Objectives

The main objectives of this chapter are:

- Analyze the computational characteristics of the FFT algorithm and identify performance bottlenecks.
- Design and implement hardware accelerators for FFT butterfly operations.
- Integrate the accelerators into the PULP platform using different hardware interfaces.
- Modify the software framework to support accelerator utilization.
- Evaluate the impact of architectural and implementation optimizations on system performance.

### 1.1.2 Chapter Contributions

The main contributions presented in this chapter include:

- The development of several FFT butterfly accelerator architectures.
- The implementation and evaluation of different radix-based FFT approaches.
- The integration of the accelerators into the PULP platform through AXI and TCDM interfaces.
- The development of the software infrastructure required for accelerator control and data movement.
- A comprehensive performance analysis of the proposed hardware–software co-design solutions.

## 1.2 FFT Computational Analysis

### 1.2.1 FFT in the MIMO-OFDM Receiver

As described in the chapter “**MIMO in Mobile Communication Systems,**” the FFT is a fundamental component of the OFDM receiver chain. Following cyclic prefix removal, the received time-domain samples are transformed into the frequency domain before channel equalization and MIMO detection can be performed [ch9][1].

The transformation is given by the Discrete Fourier Transform (DFT):

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-2\pi kn/N} \quad (1.1)$$

where:

- $x[n]$ : is the received time-domain sample,
- $X[k]$ : is the frequency-domain output at subcarrier  $k$ ,
- $N$ : is the FFT size,
- $n$ : is the time-domain sample index,
- $K$ : is the frequency-domain subcarrier index.

Direct computation of the DFT requires  $O(N^2)$  operations. The FFT reduces this complexity to  $O(N \log(N))$  through recursive decomposition into smaller operations known as butterflies [ch9][1].

### 1.3 Butterfly Computation as the Main Processing Element

The FFT achieves its computational efficiency by decomposing the DFT into a sequence of smaller operations known as butterflies. Rather than computing all frequency components directly, the FFT repeatedly combines intermediate results through butterfly computations distributed across multiple processing stages.



Figure 1.1: Butterfly Operations Distributed Across FFT Processing Stages

Regardless of the selected radix, the butterfly represents the fundamental computational element of the FFT. Each butterfly performs a combination of arithmetic operations, including additions, subtractions, and twiddle-factor multiplications. As these operations are executed repeatedly throughout the transform, butterfly processing accounts for a substantial portion of the overall FFT workload.

For an  $N$ -point FFT, the total number of butterfly computations grows with the transform size and the number of processing stages. Consequently, improvements in butterfly execution latency can have a direct impact on overall FFT performance.

This observation motivated the investigation of dedicated accelerators presented later in the chapter “**Hardware I: Algorithms**”.

### 1.4 Radix Selection and Design Space Exploration

The number of FFT stages depends on the selected radix according to [ch9] [1]

$$N_{stages} = \log_r(N) \quad (1.2)$$

where:

- $N_{stages}$ : is the number of FFT stages,
- $N$ : is the FFT size,
- $r$ : is the FFT radix.

For the 4096-point FFT considered in this work, the stage count is shown in Table 1.1.

| FFT Radix | Number of Stages |
|-----------|------------------|
| Radix-2   | 12               |
| Radix-4   | 6                |
| Radix-8   | 4                |

Table 1.1: FFT Stage Count for different radices

As shown in the previous, **increasing the FFT radix reduces the number of processing stages required to complete the transform**. Since each stage involves arithmetic operations, memory accesses, and synchronization events, reducing the stage count has the potential to improve overall execution efficiency.

However, the choice of radix also influences the structure and complexity of the corresponding butterfly operation. Consequently, the FFT radix represents an important architectural design parameter that affects both computational behavior and hardware implementation characteristics.

For this reason, multiple radix implementations were investigated in this work. The objective was to evaluate how different FFT decompositions influence performance when implemented as hardware accelerators on the PULP platform. The resulting butterfly architectures are presented later in “**Hardware I: Algorithms**” while their impact on system-level performance is evaluated in “**System Performance Analysis**.”

## 1.5 Computational and Communication Considerations

Although butterfly arithmetic constitutes a major portion of FFT execution, overall system performance is influenced by both **computation and data movement**. Hardware acceleration can reduce arithmetic latency, but the resulting performance gain also depends on the efficiency of communication between the processor, memory subsystem, and accelerator.

Table 1.2 summarizes the primary contributors to FFT execution time [ch9][1] .

| Component                    | Impact on Performance |
|------------------------------|-----------------------|
| <b>Butterfly Arithmetic</b>  | High                  |
| <b>Twiddle Factor Access</b> | Medium                |
| <b>Memory Transfers</b>      | High                  |
| <b>Control Overhead</b>      | Medium                |

Table 1.2: Primary contributors to FFT execution time.

This observation motivates the hardware-software co-design approach presented in this chapter “**HW/SW Codesign**”, where both accelerator performance and integration overhead must be considered when evaluating system-level improvements.

The interaction between these factors ultimately determines the effectiveness of hardware acceleration within the PULP platform.

## 1.6 Computational Bottlenecks

The computational cost of FFT processing is largely dominated by complex multiplications performed within butterfly operations. A complex multiplication is defined as

$$(a + jb)(c + jd) = (ac - bd) + j(ad + cb) \quad (1.3)$$

where:

- (a, b, c,) and (d) are real-valued operands,

- $(j)$  is the imaginary unit.

This operation requires four real multiplications and two real additions, making it significantly more expensive than a simple addition or subtraction [ch9] [1].

Table 2.2 summarizes the primary contributors to FFT execution time.

| Operation              | Relative Cost |
|------------------------|---------------|
| Addition/Subtraction   | Low           |
| Twiddle Factor Access  | Medium        |
| Complex Multiplication | High          |

Table 1.3: Major Contributors to FFT Computational Cost

The concentration of arithmetic complexity within the butterfly structure further motivates the use of dedicated hardware acceleration.

## 1.7 Hardware Accelerator Design

### 1.7.1 Design Objectives and Requirements

The FFT computational analysis presented in the previous section identified butterfly operations as the primary source of arithmetic complexity within the target OFDM receiver. Consequently, the objective of the hardware accelerator design was to reduce the execution time associated with butterfly computations while maintaining compatibility with the PULP platform.

Several design requirements were established prior to the accelerator development process:

- Compatibility with fixed-point arithmetic.
- Scalability to different FFT radix implementations.
- Efficient integration within the PULP architecture.
- Low communication overhead between software and hardware.
- Sufficient throughput to justify hardware acceleration.

In addition to computational performance, accelerator efficiency was evaluated from a system-level perspective. As discussed in “**Computational and Communication Consideration**” section, improvements in arithmetic latency do not necessarily translate directly into overall application speedup if communication and synchronization overheads dominate execution time.

| Requirement            | Description                    |
|------------------------|--------------------------------|
| Arithmetic Precision   | Fixed-point complex arithmetic |
| FFT Support            | Multiple radix structures      |
| Platform Compatibility | PULP integration               |
| Scalability            | Support for larger FFT sizes   |
| Performance            | Reduced butterfly latency      |
| Resource Efficiency    | Moderate area utilization      |

Table 1.4: Hardware Accelerator Design Requirements

## 1.8 Radix-2 Butterfly Architecture

The radix-2 butterfly was selected as the baseline architecture due to its simplicity and widespread use in FFT implementations. A radix-2 FFT decomposes the transform into a sequence of two-point butterfly operations, requiring  $\log_2 N$  stages to complete an (N)-point FFT [ch9] [1].

The radix-2 butterfly performs one complex multiplication followed by two complex additions according to

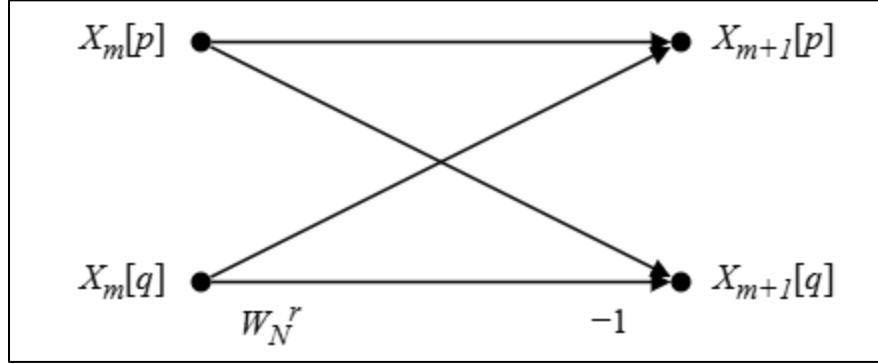


Figure 1.2: Radix-2 butterfly (adapted from [ch9][1])

$$Y_0 = A + BW \quad (1.4)$$

$$Y_1 = A - BW \quad (1.5)$$

where:

- A and B are complex input samples,
- W is the twiddle factor,
- Y0 and Y1 are the butterfly outputs.

The relatively simple arithmetic structure results in straightforward control logic and moderate hardware resource requirements. Furthermore, the butterfly can be easily pipelined, making it suitable for high-frequency implementations.

However, for a 4096-point FFT, a radix-2 decomposition requires twelve processing stages. **The large number of stages increases the amount of intermediate data movement and synchronization required during FFT execution**, which may limit the overall performance gains achievable through butterfly acceleration alone.

| Parameter               | Value |
|-------------------------|-------|
| FFT Stages (4096 point) | 12    |
| Arithmetic Complexity   | Low   |
| Control Complexity      | Low   |
| Hardware Cost           | Low   |
| Data Movement           | High  |

Table 1.5: Characteristics of the Radix-2 Butterfly



## 1.9 Radix-4 Butterfly Architecture

To reduce the number of FFT stages, a radix-4 butterfly architecture was also investigated. In a radix-4 decomposition, four input samples are processed together, reducing the number of required FFT stages **from twelve to  $\log_4 N = 6$**  [ch9][1].

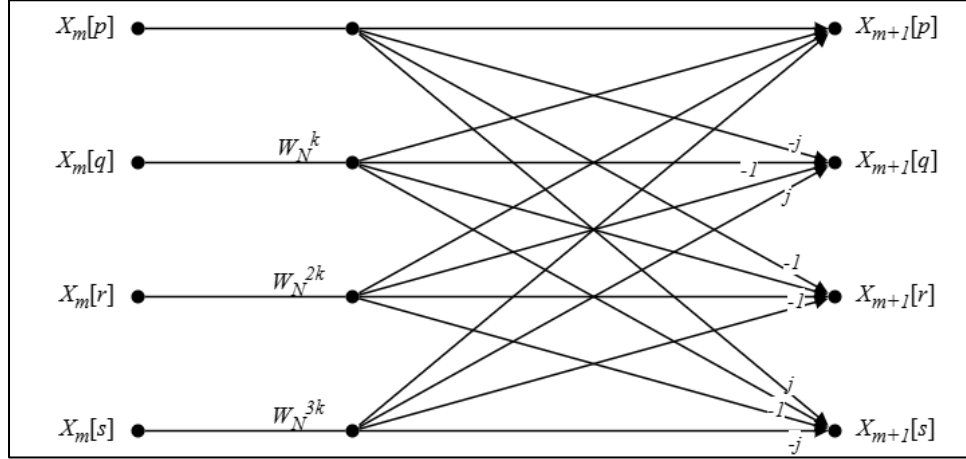


Figure 1.3: Radix-4 butterfly (adapted from [ch9][1])

By decreasing the number of stages, radix-4 FFTs reduce the amount of intermediate storage, memory traffic, and synchronization events required during execution. These characteristics make radix-4 an attractive compromise between computational efficiency and implementation complexity.

The reduction in stage count is achieved at the expense of a more complex butterfly structure. Compared to radix-2 implementations, radix-4 butterflies require additional arithmetic operations and more sophisticated control logic. Consequently, the hardware resource requirements increase, although the architecture remains practical for implementation within the target platform.

| Parameter               | Value  |
|-------------------------|--------|
| FFT Stages (4096-point) | 6      |
| Arithmetic Complexity   | Medium |
| Control Complexity      | Medium |
| Hardware Cost           | Medium |
| Data movement           | Lower  |

Table 1.6: Characteristics of the Radix-4 Butterfly

## 1.10 Radix-8 Butterfly Architecture

To further reduce the FFT stage count, a radix-8 butterfly architecture was evaluated. In this approach, eight input samples are processed within a single butterfly structure, reducing the required number of stages  **$\log_8 N$**  to **only four processing** [ch9][1].

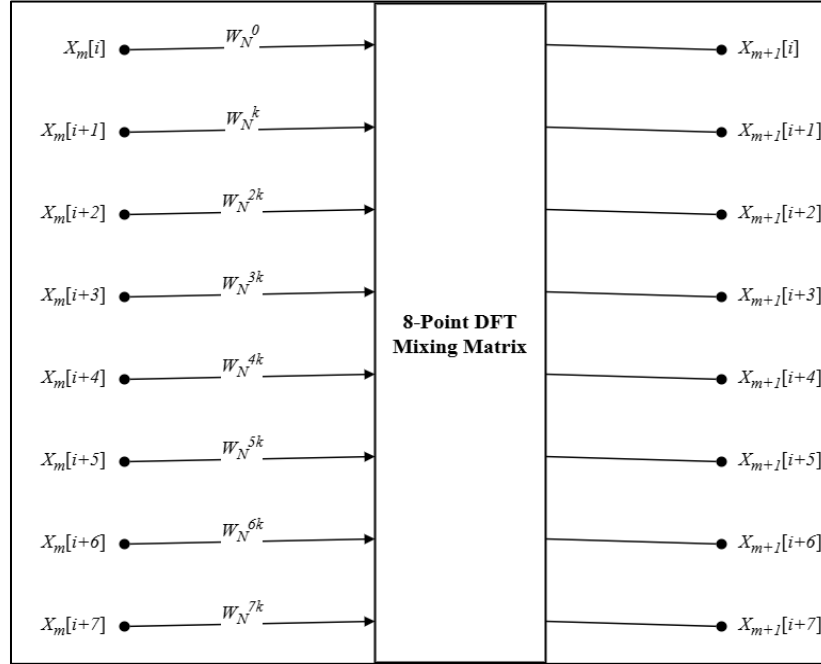


Figure 1.4: Radix-8 butterfly (adapted from [ch9][1])

The primary advantage of radix-8 FFTs is the significant reduction in stage count, which decreases synchronization overhead and limits the number of intermediate memory transactions. These properties can potentially improve system-level performance by reducing the amount of data movement required during FFT execution.

However, the radix-8 butterfly introduces substantially higher arithmetic complexity than radix-2 and radix-4 implementations. Additional arithmetic units, larger data paths, and more complex control structures are required to support the increased number of simultaneous computations. As a result, radix-8 implementations typically incur higher hardware costs and greater design complexity.

| Parameter               | Value |
|-------------------------|-------|
| FFT Stages (4096-point) | 4     |
| Arithmetic Complexity   | High  |
| Control Complexity      | High  |
| Hardware Cost           | High  |

Table 1.7: Characteristics of the Radix-8 Butterfly

## 1.11 Comparison of Investigated Radices

To evaluate the impact of FFT decomposition on accelerator performance, radix-2, radix-4, and radix-8 implementations were investigated. The three architectures provide different trade-offs between stage count, arithmetic complexity, and hardware resource utilization.

| Metric                | Radix-2 | Radix-4 | Radix-8 |
|-----------------------|---------|---------|---------|
| Stages (4096-point)   | 12      | 6       | 4       |
| Arithmetic Complexity | Low     | Medium  | High    |
| Control Complexity    | Low     | Medium  | High    |
| Hardware Cost         | Low     | Medium  | High    |

Table 1.8: Comparison of Investigated FFT Radices

As the radix increases, the number of FFT stages decreases significantly. However, this reduction is accompanied by a corresponding increase in butterfly complexity and hardware resource requirements. The performance implications of these trade-offs are evaluated later in “**System Performance Analysis.**”

## 1.12 Hardware Integration into the PULP Architecture

### 1.12.1 Overview of the PULP SoC Domain

The FFT butterfly accelerators were initially developed as standalone hardware modules. Although these accelerators successfully performed the required arithmetic operations, they could not be directly controlled by software executing on the PULP platform. Consequently, an integration mechanism was required to connect the accelerators to the system infrastructure.

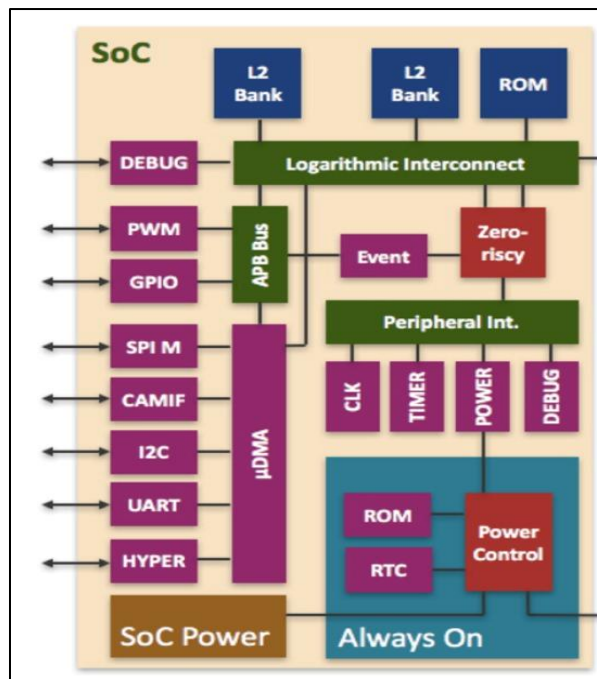


Figure 1.5: PULP SoC Domain ([ch9][2])

As described in “**PULP Platform**,” the PULP architecture combines programmable RISC-V cores with a shared memory hierarchy and a flexible communication network. Within the SoC domain, software interacts with memories and peripherals through memory-mapped transactions. This organization enables hardware components to be accessed using standard processor instructions [ch9] [2].

Communication between system components is performed through the AXI interconnect. Each peripheral is assigned a dedicated address range within the global memory map, allowing the interconnect to route transactions to the appropriate destination. This mechanism provides a standardized method for integrating custom hardware modules into the platform[ch9][2].

A key architectural characteristic of the PULP platform is the presence of two communication networks optimized for different workloads. As described in “**PULP Platform**” and “**Multi Cluster PULP**,” low-latency memory accesses are performed through the TCDM interconnect, while communication with peripherals and external hardware modules is handled through the AXI crossbar. The TCDM network is optimized for fast shared-memory transactions, whereas the AXI infrastructure provides a flexible, address-based mechanism for accessing system peripherals [ch9][2].

In the PULP communication model, hardware components are identified through dedicated memory addresses rather than explicit connections to software. Consequently, interaction with a hardware accelerator consists of reading from and writing to a predefined address range. This communication can be divided into a **control path**, used to configure and trigger accelerator execution, and a **data path**, used to transfer operands and retrieve computation results[ch9][2].

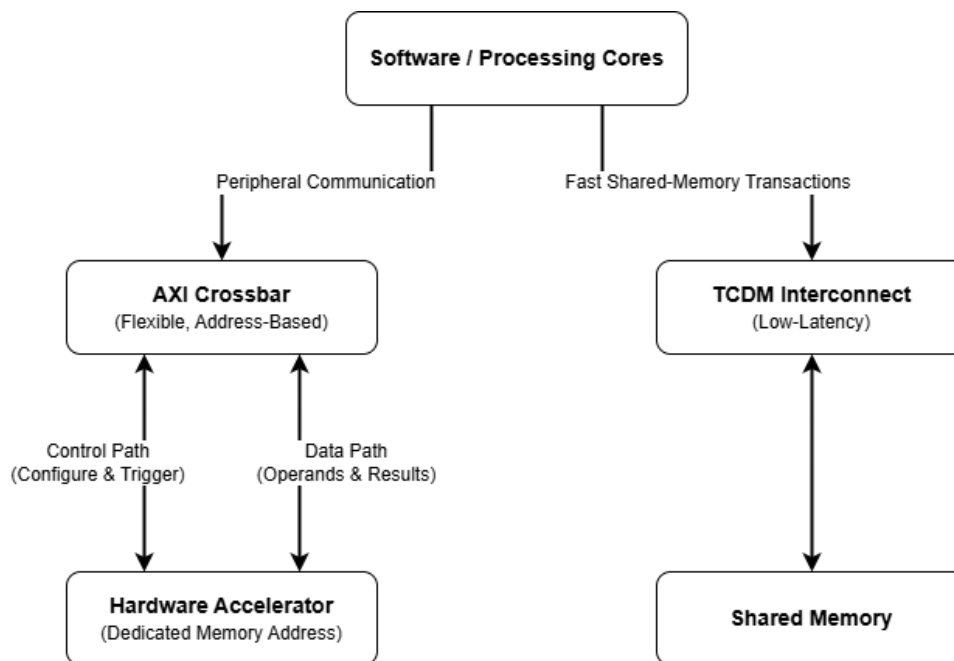


Figure 1.6: AXI and TCDM Communication Networks in the PULP Architecture

The objective of the integration process is therefore **to transform the FFT accelerator into a memory-mapped peripheral**. This step represents a key component of the hardware/software co-design methodology discussed in this chapter “**HW/SW Codesign**.”

## 1.13 AXI Slave Integration

### 1.13.1 Integration Philosophy

The FFT accelerators developed expose application-specific interfaces consisting of operand inputs, control signals, and computation outputs. While these interfaces are suitable for arithmetic processing, they are not compatible with the communication mechanisms used by software. An intermediate integration layer is therefore required[ch9][3].

To address this challenge, a wrapper-based integration approach was adopted. The accelerator is encapsulated inside a dedicated module that exposes a standard AXI slave interface toward the SoC while preserving the original data path internally. This approach avoids modifications to the computational core [ch9][3].

The wrapper establishes a clear separation between communication and computation. Bus transactions are handled by the wrapper, while arithmetic processing remains the responsibility of the accelerator. This separation simplifies both development and verification [ch9][3].

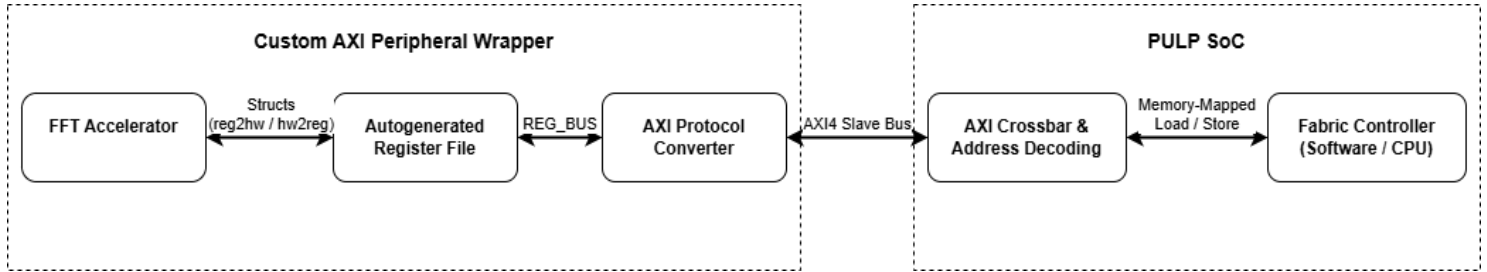


Figure 1.7: Architectural Layers of the FFT Accelerator Integration

### 1.13.2 Wrapper Architecture

The wrapper architecture is composed of three primary building blocks: the AXI interface, the register file, and the FFT accelerator. Together, these components establish a complete communication path between software and hardware. Each block performs a dedicated role within the integration process[ch9][3].

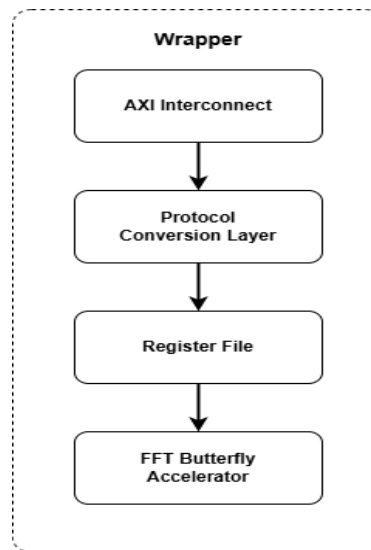


Figure 1.8: Internal organization of the accelerator wrapper. (Adapted from [ch9][3])

The AXI interface receives transactions originating from the processor and converts them into internal register accesses. These accesses may correspond to writing input operands, configuring execution parameters, or reading computation results. In this way, the complexity of the communication protocol is isolated from the accelerator.

The register file serves as the communication bridge between software and hardware. It stores configuration information, input operands, control signals, status flags, and output results. Through this register-based organization, software can interact with the accelerator using a familiar memory-mapped programming model.

The FFT accelerator remains focused exclusively on arithmetic computation. It operates on the information supplied through the register file and updates the corresponding output registers once execution is complete. This layered structure improves modularity and promotes design reuse.

| Component              | Function                                    |
|------------------------|---------------------------------------------|
| <b>AXI Interface</b>   | Receives and translates AXI transactions    |
| <b>Register File</b>   | Stores control, status, inputs, and outputs |
| <b>FFT Accelerator</b> | Performs butterfly computations             |

Table 1.9: Internal organization of the accelerator wrapper.

### 1.13.3 Register-Based Communication and Datapath Connectivity

Communication between software and the accelerator is performed through memory-mapped registers. Before execution begins, software writes the required operands and configuration parameters into dedicated input registers. A control register is subsequently used to initiate the accelerator operation[ch9][3].

Once execution starts, the accelerator processes the supplied data independently of the processor. During this period, software may monitor execution progress through status registers. These registers provide a simple synchronization mechanism between the processor and the hardware accelerator.

After completion, the generated outputs are stored in dedicated result registers. Software retrieves these values through standard memory read operations. From the processor perspective, interacting with the accelerator is therefore equivalent to accessing any other peripheral within the system [ch9][3].

| Register Type           | Purpose                             |
|-------------------------|-------------------------------------|
| <b>Control</b>          | Execution control and configuration |
| <b>Status</b>           | Completion and execution state      |
| <b>Input Registers</b>  | Butterfly operands                  |
| <b>Output Registers</b> | Computation results                 |

Table 1.10: Accelerator Register Categories

The register file establishes the communication path between software and the FFT accelerator. Configuration parameters and input operands are transferred from the processor to the accelerator through dedicated registers. Similarly, results and status information are propagated from the accelerator back to software[ch9][3].

This organization allows the accelerator to remain entirely focused on arithmetic processing. Communication, synchronization, and protocol-related functionality are handled by the surrounding integration infrastructure. As a result, the computational core remains independent of system-level communication details.

An additional advantage of this approach is the separation of responsibilities between software and hardware. The processor manages data movement and execution control, while the accelerator performs the computational task. This division simplifies both debugging and verification activities.

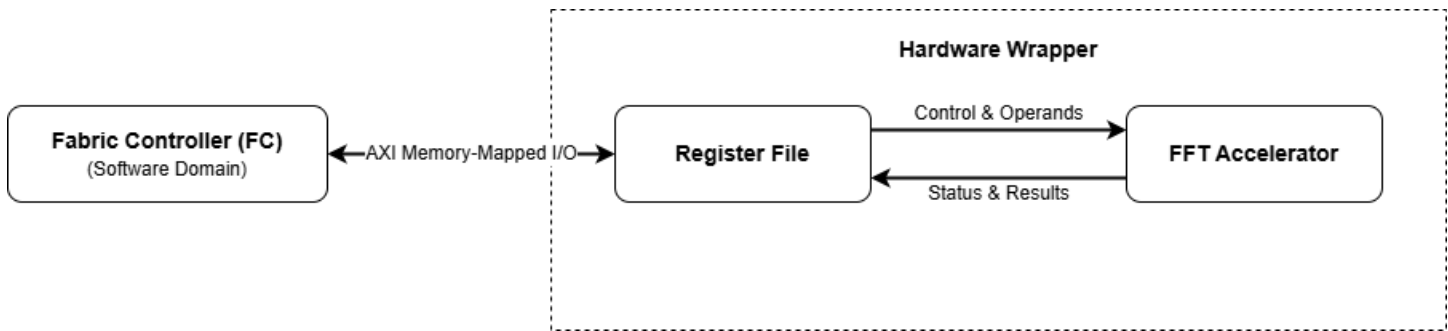


Figure 1.9: Data flow between software, register file, and FFT accelerator.

### 1.13.4 SoC-Level Integration

Once the wrapper architecture is completed, the accelerator can be connected to the SoC communication infrastructure. At this stage, the accelerator becomes a standard peripheral attached to the AXI interconnect. This allows software-generated transactions to reach the accelerator [ch9][3].

Integrating a new peripheral requires extending the interconnect with an additional AXI slave connection. A dedicated communication path is established between the accelerator wrapper and the AXI crossbar. This connection enables the accelerator to participate in the same communication framework used by the other peripherals in the system. However, physical connectivity alone is insufficient. The interconnect must also be capable of identifying which transactions are intended for the accelerator. This is provided through memory mapping and address decoding mechanisms[ch9][3].

### SoC Interconnect

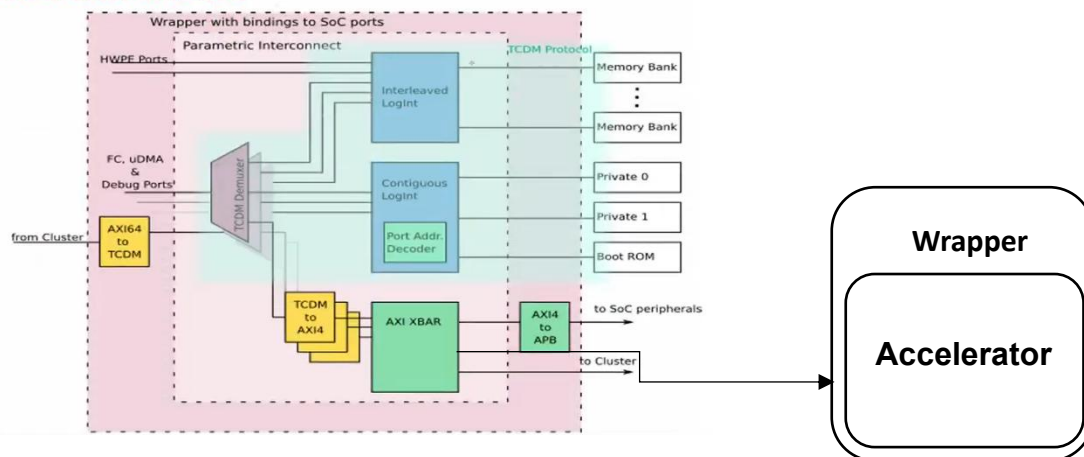


Figure 1.10: SoC Interconnect with the Integrated Accelerator([ch9][2])

### 1.13.5 Memory Mapping and Address Decoding

To make the accelerator accessible from software, a dedicated address range is assigned within the global SoC memory map. This address range uniquely identifies the peripheral and serves as the destination for all accelerator-related transactions. Care must be taken to ensure that the selected region does not overlap with existing memory or peripheral spaces [ch9][3].

The AXI interconnect maintains a set of address decoding rules that associate address regions with specific peripherals. Whenever the processor issues a memory transaction, the interconnect examines the target address and determines the appropriate destination. Transactions targeting the accelerator region are automatically routed to the accelerator wrapper [ch9][3].

From the software perspective, the FFT accelerator therefore appears as a conventional memory-mapped peripheral. All configuration, control, and result retrieval operations are performed through standard memory accesses. This preserves the programming model of the PULP platform while enabling hardware acceleration.

The integration methodology described in this section transforms the FFT accelerator into a software-accessible hardware peripheral. This provides the foundation for the software interaction mechanisms.

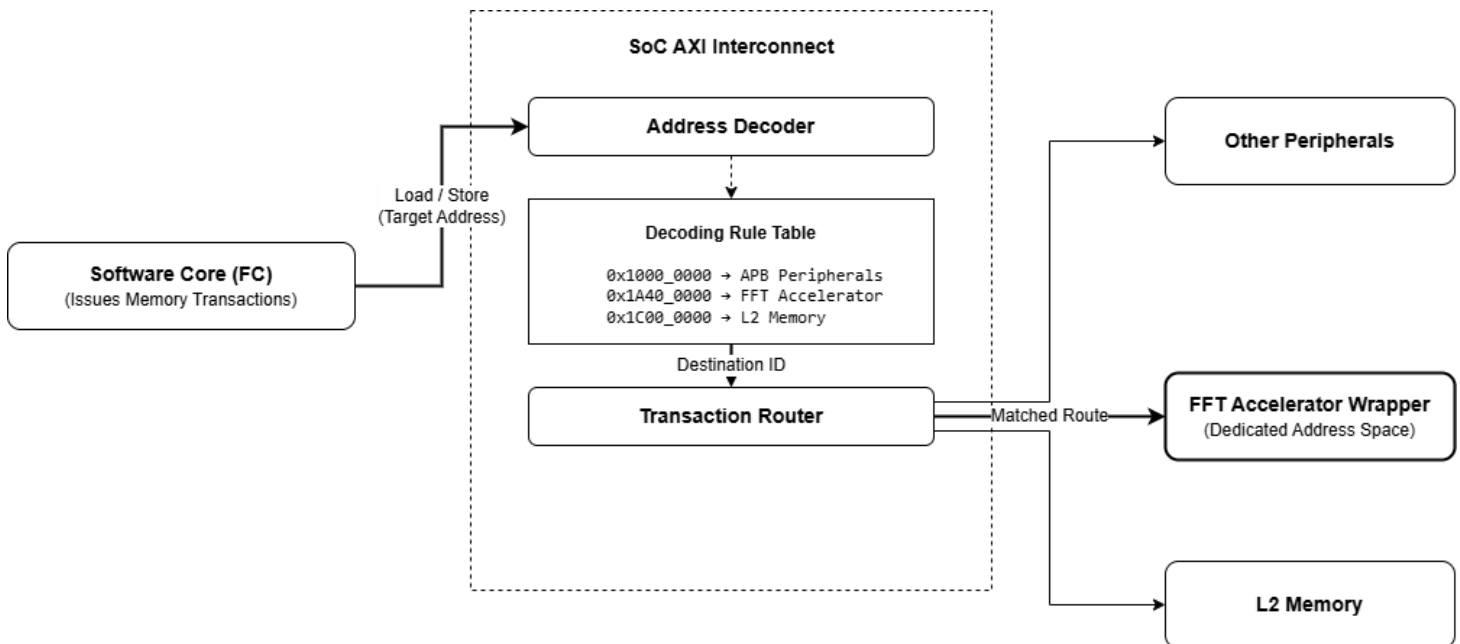


Figure 1.12: AXI address decoding and transaction routing within the SoC interconnect.



## 2 References

- [1] A. V. Oppenheim and R. W. Schaffer, *Discrete-time signal processing*, 3rd ed. Upper Saddle River: Pearson, 2010.
- [2] D. Schiavone, “Understanding and working with PULP,” [Online]. Available: [https://www.pulp-platform.org/pulp\\_training.html](https://www.pulp-platform.org/pulp_training.html)
- [3] R. Balas and M. Eggimann, “A Deep Dive into HW/SW Development with PULP,” [Online]. Available: [https://www.pulp-platform.org/pulp\\_training.html](https://www.pulp-platform.org/pulp_training.html)